

5-Concurrency

Tutorium 24

Siebo-Ordner für alle, die regulär nicht in TU 24 sind:

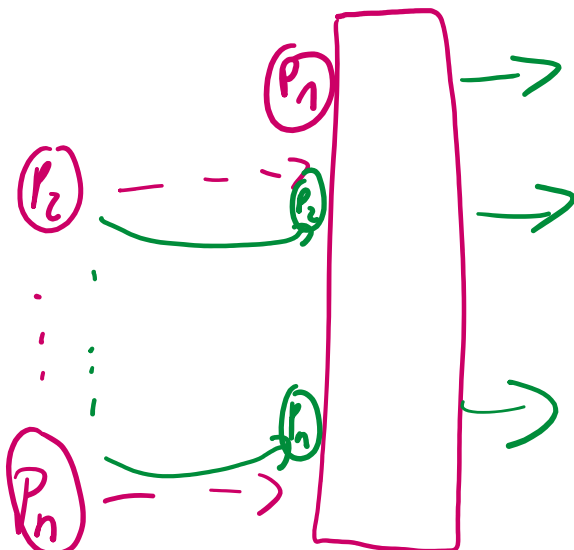


Aufgabe: Definiert je einen struct für...

- a. Mutex
- b. Barrier

unter Verwendung von Semaphoren.

n Prozesse



```

1  struct Mutex {
2      sem_t mutex;
3  };
4
5  void mutex_init(struct Mutex *mutex) {
6      sem_init(&mutex->mutex, 0, 1); //Mutex unlocked
7  }
8
9  void mutex_lock(struct Mutex *mutex) {
10     sem_wait(&mutex->mutex);    //Mutex-- : 1->0
11 }
12
13 void mutex_unlock(struct Mutex *mutex) {
14     sem_post(&mutex->mutex);    //Mutex++ : 0->1
15 }

```

```

struct Barrier {
    int n_threads;
    int count;
    sem_t barrier;
    sem_t mutex;
};

// initialize the barrier for later use
void barrier_init(struct Barrier *barrier, int n_threads) {
    barrier->count = 0;
    barrier->n_threads = n_threads;
    sem_init(&barrier->mutex, 0, 1);
    sem_init(&barrier->barrier, 0, 0);
}

// Barrier to ensure all threads wait here before being released
void barrier_wait(struct Barrier *barrier) {
    //Mutex beanspruchen
    sem_wait(&barrier->mutex);
    barrier->count++;

    if (barrier->count == barrier->n_threads) {
        //Alle Prozesse sind an Barriere angekommen, der erste kann wieder freigegeben werden
        sem_post(&barrier->mutex);
        sem_post(&barrier->barrier);
        return;
    }

    //mutex wieder freigeben, nächste Prozess kann ausgeführt werden
    sem_post(&barrier->mutex);

    sem_wait(&barrier->barrier);
    sem_post(&barrier->barrier);
}

```

Task 2a

Freitag, 28. Juni 2024 10:28

```
1 #define KLASSIKER 0
2 #define VEGETARIAN 1
3
4 int studentsAtCounter = 0;
5 int klassiker = 0;
6 int vegetarian = 0; // shared variables
7
8 void fillKlassiker(int n) {
9     klassiker += n;
10 }
11
12 void fillVegetarian(int n) {
13     vegetarian += n;
14 }
15
16 void arrivalStudent(int wish) {
17     while (studentsAtCounter > 0);
18     studentsAtCounter++;
19     if (wish == KLASSIKER) {
20         while (klassiker == 0) {
21             // wait
22         }
23         klassiker--;
24     }
25     else if (wish == VEGETARIAN) {
26         while (vegetarian == 0) {
27             // wait
28         }
29         vegetarian--;
30     }
31     studentsAtCounter--;
32 }
```

Dieses Programm simuliert den Ablauf der Essensausgabe in der Mensa Ahornstraße. Der Einfachheit halber gibt es hier nur Klassiker und Vegetarisch.

An der Ausgabe sind beide Gerichte verfügbar. Es kann nur eine Person gleichzeitig ihren Essenswunsch äußern. Falls das Gericht verfügbar ist, wird der Person eine Portion ausgehändigt, die Person bezahlt und verlässt die Ausgabe wieder.

Sollte das Gericht nicht verfügbar sein, wartet die Person bis eine neue Portion dieses Gerichts fertig ist. Die neuen Portionen werden vom Mensapersonal freigegeben.

Die Funktion `arrivalStudent()` wird von einem anderen Prozess aufgerufen, `fillKlassiker()` und `fillVegetarian()` können nur von einem Prozess aufgerufen werden.

Aufgabe: Erkläre den Begriff "Busy waiting" anhand dieses Programms und nenne 2 Beispiele aus dem Pseudocode links.

Busy Waiting ist, wenn ein Prozess andauernd eine Bedingung abfragt, ohne dabei produktive Arbeit auszuführen. Dabei wird die CPU ausgelastet, ohne eine sinnvolle Aufgabe abzuarbeiten --> CPU-Verschwendung. Beispiele sind links markiert.

Task 2b

Freitag, 28. Juni 2024 10:39

```
1  #define KLASSIKER 0
2  #define VEGETARIAN 1
3
4  int studentsAtCounter = 0;
5  int klassiker = 0;
6  int vegetarian = 0; // shared variables
7  Mutex counterLock; // initialize mutex
8  Mutex klassikerLock; // initialize mutex
9  Mutex vegetarianLock; // initialize mutex
10
11 void fillKlassiker(int n) {
12     mutex_lock(&klassikerLock);
13     klassiker += n;
14     mutex_unlock(&klassikerLock);
15 }
16
17 void fillVegetarian(int n) {
18     mutex_lock(&vegetarianLock);
19     vegetarian += n;
20     mutex_unlock(&vegetarianLock);
21 }
22
23 void arrivalStudent(int wish) {
24     // ensure only 1 student at a time
25     mutex_lock(&counterLock);
26     studentsAtCounter++;
27     if (wish == KLASSIKER) {
28         while (1) {
29             mutex_lock(&klassikerLock);
30             if (klassiker > 0) {
31                 klassiker--;
32                 mutex_unlock(&klassikerLock);
33                 break;
34             }
35             mutex_unlock(&klassikerLock);
36         }
37     } else if (wish == VEGETARIAN) {
38         while (1) {
39             mutex_lock(&vegetarianLock);
40             if (vegetarian > 0) {
41                 vegetarian--;
42                 mutex_unlock(&vegetarianLock);
43                 break;
44             }
45             mutex_unlock(&vegetarianLock);
46         }
47     }
48     studentsAtCounter--;
49     mutex_unlock(&counterLock);
50 }
```

Die Critical Sections sind um die Shared Variables (studentsAtCounter, klassiker,

vegetarian platziert, um diese vor Race Conditions zu schützen. Allerdings wird hier immer noch Busy Waiting benutzt und so CPU Zeit verschwendet.

Task 2c

Freitag, 28. Juni 2024 10:53

```
1  #define KLASSIKER 0
2  #define VEGETARIAN 1
3
4  Semaphore klassikerSem(0); // initialize semaphore to 0: no klassiker available at start
5  Semaphore vegetarianSem(0); // initialize semaphore to 0: no vegetarian available at start
6  Semaphore counter(1); // initialize semaphore to 1: one place available at the counter
7
8  void fillKlassiker(int n) {
9      for (int i = 0; i < n; i++) {
10         sem_post(&klassikerSem);
11     }
12 }
13
14 void fillVegetarian(int n) {
15     for (int i = 0; i < n; i++) {
16         sem_post(&vegetarianSem);
17     }
18 }
19
20 void arrivalStudent(int wish) {
21     // ensure only 1 student at a time at the counter
22     sem_wait(&counter);
23     if (wish == KLASSIKER) {
24         sem_wait(&klassikerSem);
25     } else if (wish == VEGETARIAN) {
26         sem_wait(&vegetarianSem);
27     }
28     sem_post(&counter);
29 }
```

Die Semaphoren werden genutzt um die Verfügbarkeit der Gerichte zu managen. Immer wenn neue Portionen hinzugefügt werden wird `sem_post()` aufgerufen. Durch `sem_wait()` wird realisiert, dass die Studierenden auf ihre Portion warten, wenn sie nicht verfügbar ist. Dadurch wird Busy Waiting verhindert, da Semaphore mit Signaling arbeiten.

Task 3

Freitag, 28. Juni 2024 11:13

Process A:

```
1 while (condition) {  
→ 2   sem_wait(hardDrive);  
3   sem_wait(cardReader);  
4   useDevices();  
5   sem_post(hardDrive);  
6   sem_post(cardReader);  
7 }
```

Gegeben sind 3 Ressourcen: Hard drive, Card reader und Printer.

Jeder der Prozesse braucht **genau 2** dieser Ressourcen im **exklusiven Zugriff**.

Links sind die Prozessabläufe definiert. Dabei sind alle Semaphoren auf 1 initialisiert.

Process B:

```
1 while (condition) {  
→ 2   sem_wait(printer);  
3   sem_wait(hardDrive);  
4   useDevices();  
5   sem_post(printer);  
6   sem_post(hardDrive);  
7 }
```

Funktionieren die Abläufe wie gewünscht?
Falls es Fehler oder potenzielle Probleme gibt, behebt diese bzw. beugt diesen vor.

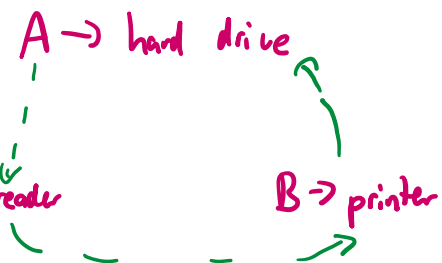
Process C:

```
1 while (condition) {  
→ 2   sem_wait(cardReader);  
3   sem_wait(printer);  
4   useDevices();  
5   sem_post(cardReader);  
6   sem_post(printer);  
7 }
```

Hier kann potenziell ein Deadlock entstehen:

Alle vier Bedingungen sind erfüllt:

- Mutual Exclusion
 - Hold and wait
 - Non-preemption
 - Circular wait
- durch Aufgabenstellung erfüllt



Lösung: Ablauf des Ressourcenzugriffs definieren:

hard drive → printer → card reader

Process A:

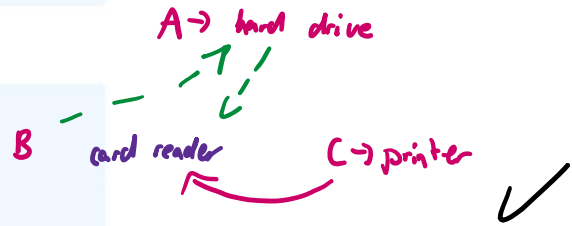
```
1 while (condition) {
2   sem_wait(hardDrive);
3   sem_wait(cardReader);
4   useDevices();
5   sem_post(hardDrive);
6   sem_post(cardReader);
7 }
```

Process B:

```
1 while (condition) {
2   // switch the locks to respect the order
3   sem_wait(hardDrive);
4   sem_wait(printer);
5   useDevices();
6   sem_post(hardDrive);
7   sem_post(printer);
8 }
```

Process C:

```
1 while (condition) {
2   // switch the locks to respect the order
3   sem_wait(printer);
4   sem_wait(cardReader);
5   useDevices();
6   sem_post(printer);
7   sem_post(cardReader);
8 }
```



Task 4

Freitag, 28. Juni 2024 11:23

Beim Sleeping Barber Problem haben wir einen einzelnen Barber in einem Barbershop mit 20 Warteplätzen. Es kann immer nur genau ein Kunde vom Barber bedient werden. Wenn keine Kunden im Barbershop sind, schläft der Barber. Der nächste Kunde, der den Shop betritt, muss den Barber dann aufwecken.

Die Aufgabe ist es nun, Routinen für den Barber und den Kunden zu schreiben und dabei Synchronisationsmechanismen zu benutzen.

Anforderungen:

- Der Barber schläft, wenn der Wartebereich leer ist.
- Ein Kunde weckt den Barber auf, falls er gerade schläft.
- Es kann nur ein Kunde gleichzeitig bedient werden
- Der Wartebereich kann maximal 20 Kunden aufnehmen.

Fragen:

- Welche Konstanten bzw. Variablen müssen deklariert werden?
- Wie viele Semaphoren müssen wir nutzen und was sind deren Werte bei Initialisierung?
- Wie viele Mutexe werden genutzt?

```
1 struct barbershop {
2     int chairs = 20;
3     int waiting;
4     sem_t customers;
5     sem_t barbers;
6     sem_t mutex;
7 }
8
9 void barber_routine(struct barbershop *shop) {
10     while (1) {
11         //Keinen Kunden -> Barber schläft
12         //Wenn doch, dann:
13         sem_wait(&shop->customers); // P > 0
14
15         //Kunde kommt rein -> Variable bearbeiten, also Mutex beanspruchen (Variable wird gesperrt)
16         sem_wait(&shop->mutex);
17         shop->waiting--;
18         sem_post(&shop->mutex);
19
20         //Kunde kann frisiert werden
21         sem_post(&shop->barbers); // P ++ , also auf 1
22         cut_hair(shop);
23     }
24 }
25
26 void customer_routine(struct barbershop *shop) {
27     sem_wait(&shop->mutex);
28     //Schauen, ob noch Warteplätze frei sind, wenn nicht gehen wir
29     if (shop->waiting < shop->chairs) {
30         //wir setzen uns hin -> Ein Kunde mehr wartet
31         shop->waiting++;
32         sem_post(&shop->customers); // ein kunde mehr in der warteschlange
33         //Warten, bis wir an der reihe sind
34         sem_post(&shop->mutex);
35         sem_wait(&shop->barbers); // P -- , also auf 0
36         get_haircut(shop);
37     }
```

```

26 void customer_routine(struct barbershop *shop) {
27     sem_wait(&shop->mutex);
28     //Schauen, ob noch Warteplätze frei sind, wenn nicht gehen wir
29     if (shop->waiting < shop->chairs) {
30         //wir setzen uns hin -> Ein Kunde mehr wartet
31         shop->waiting++;
32         sem_post(&shop->customers); // ein kunde mehr in der warteschlange
33         //Warten, bis wir an der reihe sind
34         sem_post(&shop->mutex);
35         sem_wait(&shop->barbers); // P -- , also auf 0
36         get_haircut(shop);
37     }
38     } else { //wir gehen
39         sem_post(&shop->mutex);
40     }
41 }

```